La API y el contenedor (S2)

De un modelo guardado en disco a un servicio que otra persona puede llamar: una API HTTP, una imagen Docker y un primer workflow de GitHub Actions que comprueba ambos en cada cambio.

```
 pipeline (S1)                          API (S2)                          imagen
 ml_dataset ─► train.py ─► models/modelo_mlflow/  ──carga──► ia_mlops/api/ ──copia──►
Dockerfile ─► contenedor
                          models/metadatos.json              (uvicorn)
:8080
```

Cómo se usa

```
python -m ia_mlops.pipeline                    # entrena y deja models/modelo_mlflow/ y
models/metadatos.json
make api                                       # o:  python -m uvicorn ia_mlops.api.app:app -
-port 8080
```

Abre http://127.0.0.1:8080/docs: FastAPI genera la documentación interactiva a partir del contrato.

```
curl http://127.0.0.1:8080/listo
curl -X POST http://127.0.0.1:8080/predecir -H "content-type: application/json" -d
"@tests/datos/cliente_ejemplo.json"
```

Con las comillas dobles alrededor de `@ruta` el mismo comando sirve en Linux, macOS y CMD. En PowerShell escribe `curl.exe` (a secas, `curl` es otro comando) y mantén las comillas: sin ellas, PowerShell interpreta la `@`. *Windows: sin probar en esta versión del curso.*

La API sirve el modelo que deja el **paquete** (`python -m ia_mlops.pipeline`) o el cuaderno 04 con USAR_MLFLOW en `True` o `False`: ambos guardan `models/modelo_mlflow/`. Si falta, `/listo` responde 503 y dice qué ejecutar.

Endpoints

Método y ruta	Para qué	Respuesta
`GET /salud`	¿El proceso está vivo? No mira el modelo.	200 `{"estado":"ok"}`
`GET /listo`	¿El modelo está cargado y coincide con su contrato?	200, o **503** con el motivo
`GET /modelo`	Qué modelo es: huella de los datos, variables, métricas de validación, versiones	200, o 503
`POST /predecir`	Probabilidad de recompra de **un** cliente	200 con `probabilidad`, `modelo`, `huella_datos`, `advertencias`
`POST /predecir/lote`	Lo mismo para hasta 1 000 clientes (`{"clientes":[…]}`), en el mismo orden	200

`/salud` y `/listo` son cosas distintas: un servicio puede estar vivo y no estar listo (arrancó pero no encuentra el modelo). Un orquestador usa `/salud` para decidir si **reinicia** el proceso y `/listo` para decidir si le **envía tráfico**.

El contrato

Lo que entra son las 13 variables de la capa FS de un cliente en una fecha de corte (la misma lista que `config.FEATURES`). Lo define `ia_mlops/api/esquemas.py` y FastAPI lo convierte en validación y documentación.

Qué ocurre	Código	Ejemplo
El cuerpo cumple el contrato	200	`tests/datos/cliente_ejemplo.json`
Falta una variable, el tipo no es el que toca, o el valor es imposible	**422**	`recencia_dias: -1`, `es_reino_unido: 2`, `frecuencia: 0`
Llega un campo que no es del contrato	**422**	`y`, `cliente_id`, `fecha_corte`: se rechazan en vez de ignorarse en silencio
Combinación que FS nunca produce	**422**	más facturas en 90 días que en total; `ticket_medio` que no es `monto_total / frecuencia`; primera compra posterior a la última
El valor es posible pero el modelo **no lo vio** al entrenar	200 con `advertencias`	`monto_total` mayor que el máximo de train
Falta `dias_medios_entre_compras` y hay una sola factura	200 con `advertencias`	el modelo lo imputa con la mediana de train
No hay modelo, o su firma no coincide con `metadatos.json`	**503**	`/listo` explica por qué
Fallo interno	**500** sin detalles	el detalle (traceback) va al log, con el mismo `id_peticion` que ve quien llamó

Rechazar o avisar. Se rechaza lo que es *imposible* o *incoherente* (casi seguro un error de quien llama). Se avisa de lo que es *posible pero sin respaldo* en el entrenamiento: rechazarlo dejaría sin predicción a un cliente real; callarlo escondería que el modelo extrapola. Los límites de cada campo salen de **cómo se construye FS** (`sql/fs/02_poblar_fs.sql`), no de los valores vistos al entrenar.

Por qué importa validar. El modelo ganador aplica `log1p` a las variables. Con `recencia_dias = -5`, `log1p(-5)` no es un número: el error aparecería dentro de scikit-learn como un 500 incomprensible, o peor, como una probabilidad sin sentido. La validación convierte eso en un 422 que dice qué campo falló.

Sin historia no hay predicción. `frecuencia >= 1`: un cliente sin compras antes del corte no tiene fila en FS. La API no inventa valores para él; es una decisión de negocio qué hacer con los clientes nuevos.

El contrato sale de la firma del modelo

`train.py` guarda el modelo en formato MLflow (`models/modelo_mlflow/`): una carpeta que lleva dentro, además del modelo, su **firma** (nombre, tipo y obligatoriedad de cada variable de entrada) y un ejemplo.

- Al arrancar hay **dos** comprobaciones. `Servicio.cargar()` compara los nombres y el orden de la firma con `models/metadatos.json`. Después, `Servicio.comprobar_esquema()` compara los campos del esquema `Cliente` con la firma. Si algo no coincide, el servicio **no queda listo**: `/listo` y `/predecir` responden 503 con un mensaje que nombra los campos que faltan o sobran. Es mejor fallar al desplegar que devolver probabilidades con las columnas cambiadas, o un 500 en cada petición.
- Es lo que ocurre cuando se reentrena con una variable nueva y no se actualiza `ia_mlops/api/esquemas.py`: el modelo la pide y la API no la conoce. (Es el caso de la tarea.)
- Las pruebas comprueban tres cosas: que el esquema Pydantic coincide con la firma (nombres, orden, **tipos** y obligatoriedad: la firma marca `dias_medios_entre_compras` como opcional y la API también); que las 990 filas de FS de la muestra cumplen el contrato; y que las probabilidades de la API son **idénticas** (tolerancia 1e-12) a las del paquete, las del `joblib` y las del modelo MLflow, para 200 filas reales de test.
- **Por qué no `joblib` solo.** Con `joblib` el contrato viviría fuera del modelo y podría desincronizarse. En formato MLflow viaja con él.
- **Riesgo conocido.** El formato `cloudpickle` puede ejecutar código al cargarse. Aquí es aceptable porque el artefacto lo construye nuestro propio pipeline; nunca se debe cargar un modelo de origen desconocido. En S5 se documenta junto con la cadena de confianza.
- **Límite.** La API carga con `mlflow.sklearn.load_model`, que **no impone** el esquema de entrada al predecir; por eso el esquema Pydantic y la comparación de arranque son los que vigilan el contrato.

Comparar los caminos hacia el modelo

El mismo modelo se puede ejecutar por cuatro caminos: el paquete, el archivo `joblib`, el formato MLflow y la API (local o en un contenedor). Si algo cambia entre entrenar y servir (otra versión de una librería, otro orden de columnas, otro preproceso), las probabilidades dejan de coincidir. `comparar` lo mide con filas reales de la partición test:

```
python -m ia_mlops.modeling.comparar                              # paquete, joblib y MLflow
python -m ia_mlops.modeling.comparar --url http://127.0.0.1:8080  # además, la API en marcha
make comparar URL=http://127.0.0.1:8080                           # lo mismo con make
```

Opciones: `--n` (filas, por defecto 100) y `--tolerancia` (por defecto 1e-9). Código de salida: **0** todo coincide, **1** hay diferencias, **2** no se pudo comparar (sin modelo, sin base de datos o la API no responde). Con la muestra y 200 filas, la diferencia máxima entre caminos fue 0.000e+00.

Los registros (logs)

Cada petición escribe **una línea JSON por la salida estándar**:

```
{"severity": "INFO", "message": "POST /predecir 200", "evento": "peticion", "id":
"26ac527aa211", "metodo": "POST", "ruta": "/predecir", "codigo": 200, "ms": 45.6}
```

- `severity` y `message` son los campos que Cloud Logging reconoce; el resto quedan como campos consultables.
- Cada respuesta lleva `X-Request-ID` (el que envió quien llama o uno nuevo); sirve para unir lo que ve el cliente con el log.
- **Nunca se registran los datos del cliente**: solo método, ruta, código, duración e identificador.
- Nivel con `CURSO_LOG_NIVEL` (por defecto `INFO`).

Una lección de la solución de referencia. Una primera versión usaba `logging` sin configurar un manejador. Las pruebas con `TestClient` mostraban las líneas, pero con `uvicorn` de verdad **no salía ninguna**: uvicorn solo configura sus propios registros. Por eso hay una prueba que arranca el servidor como lo hace el contenedor y lee su salida (`test_con_uvicorn_de_verdad_las_lineas_json_llegan_a_stdout`).

La imagen Docker

`deploy/docker/Dockerfile`, con el contexto en la carpeta del proyecto:

```
make pipeline                                  # el modelo se entrena ANTES; la imagen solo lo
sirve
docker build -f deploy/docker/Dockerfile -t retencion-api:local .
docker run --rm -p 8080:8080 retencion-api:local
```

Docker no se instala en tu equipo: se usa en **Codespaces** (ver `03-Entorno/`). Dentro del Codespace, el archivo `.devcontainer/devcontainer.json` de este proyecto pide Python 3.11 y Docker (*sin probar*: ver «Verificación»).

Si lanzas `curl` justo después de `docker run`, puede salir `Connection reset by peer`: el puerto ya está publicado, pero `uvicorn` aún no ha arrancado. Espera unos segundos y repite (el workflow lo hace en un bucle). Tamaño de la imagen construida en GitHub Actions: **826 MB** en disco y 183 MB de «contenido» según `docker image ls`.

Decisión	Por qué
`python:3.11-slim`	Misma versión de Python que `environment.yml`.
Primero `requirements-api.lock`, después el código	La capa de dependencias solo se reconstruye si cambian las versiones: una línea de código no reinstala todo.
`requirements-api.lock` (versiones **exactas**, 53 paquetes) y no `requirements-api.txt`	Dos construcciones dan las mismas versiones. Se regenera con `make lock`.
`mlflow-skinny` y no `mlflow`	Basta para leer un modelo y su firma; no trae servidor ni SQL.
Solo `ia_mlops/`, `models/modelo_mlflow/` y `models/metadatos.json`	La imagen contiene lo que sirve y nada más: sin datos, sin SQL, sin pruebas, sin la base SQLite.
`.dockerignore`	Deja fuera datos, cuadernos, pruebas, `.git` y los secretos habituales (`.env`, `*.key`). Ojo: un patrón como `*.pyc` solo vale en la raíz; para cualquier carpeta hay que escribir `**/*.pyc`.

Decisión	Por qué
Usuario `10001`, no root	Si alguien explota la API, no es administrador del contenedor.
`HEALTHCHECK` sobre `/salud`	El motor de contenedores sabe si el proceso responde.
`CURSO_MODELOS=/app/models`	Dónde buscar el modelo dentro de la imagen.

El primer workflow

`deploy/github-actions/ci.yml` es una **plantilla**: copia el archivo a `.github/workflows/ci.yml` de tu repositorio (cuya raíz es la carpeta del proyecto). En cada `push` y `pull_request` a `develop` o `main`:

1. Instala las dependencias de `requirements-dev.txt` (las mismas versiones que `environment.yml`).
2. Ejecuta el pipeline completo con el código Python.
3. Pasa las pruebas.
4. Construye la imagen.
5. Arranca el contenedor y comprueba `/listo`, `/modelo`, una predicción y que la API **rechaza** un campo que no es del contrato (422).
6. Si algo falla, muestra el registro del contenedor.

Permisos mínimos (`contents: read`), sin secretos y sin publicar nada: publicar y desplegar llegan en S4.

Verificación

Probado

- Las 112 pruebas del proyecto base (108 rápidas y 4 lentas, que ejecutan los cuadernos) en Linux, con el entorno `env-ia-mlops`.
- Que las pruebas **detectan** fallos: `tests/mutaciones_s2.py` rompe el código de 24 maneras distintas (la API acepta campos desconocidos, no se avisa de valores fuera de rango, la app no configura el registro, la imagen corre como root…) y comprueba que cada una hace fallar alguna prueba.
- El Dockerfile y el workflow, **en GitHub Actions** (Ubuntu con Docker): construye la imagen, el contenedor responde en `/listo`, `/modelo` y `/predecir`, rechaza un campo fuera del contrato con 422 y corre con el usuario `10001`. Tres ejecuciones en verde (ids 42729626415, 42730202500 y 42730743692). La probabilidad del cliente de ejemplo es **0.8278889191932133** en local y en el contenedor, idéntica.
- El workflow pasa `actionlint` sin avisos.

Sin probar

- `.devcontainer/devcontainer.json`: no se ha abierto ningún Codespace con él. Antes de la clase, abre uno y ejecuta `docker version`.
- `docker build` y `docker run` en una máquina propia con Docker: no hay Docker en el entorno donde se preparó el curso, así que la construcción solo se ha visto en GitHub Actions.
- Los comandos de Windows (PowerShell y CMD).

Ten presente al reproducirlo: el Dockerfile se verificó con el proyecto tal como estaba cuando se ejecutó el workflow; si cambias archivos que la imagen copia (`ia_mlops/`, `models/`), repite la construcción.